



TRƯỜNG ĐẠI HỌC

SƯ PHẠM KỸ THUẬT TP.HỒ CHÍ MINH

HCMC University of Technology and Education

Bài 6: Class





Nội dung:

- Class
- Phương thức/hành động trong class

```
student1_name = "Thien Duong"
student1_age = 31
student1_math_score = 9
student1_literature_score = 8

print(student1_name)
print(student1_age)
print(student1_math_score)
print(student1_literature_score)

student2_name = "Enzo"
student2_age = 31
student2_math_score = 9
student2_literature_score = 8

print(student2_name)
print(student2_age)
print(student2_math_score)
print(student2_literature_score)
```

- Trong trường hợp có nhiều hơn 2 trường hợp, để in thông tin tương tự thì cần phải lập trình nhiều dòng lệnh hơn
- Để có thể lập trình ngắn gọn và logic hơn, ta có thể dùng **class**. **Class** là một tập lớn chứa thông tin nhỏ hơn. Ví dụ: **class student** sẽ chứa các thông tin như: **họ tên, tuổi, lớp học, điểm số....**

1. Class trong Python

Trong Python, **class (lớp)** là một **mô hình (blueprint)** để tạo ra các **đối tượng (objects)**. Lập trình hướng đối tượng giúp **quản lý dữ liệu và chức năng dễ dàng hơn** bằng cách gói gọn chúng vào một đối tượng.

- ◆ Class: Một khuôn mẫu chứa thuộc tính (attributes) và hàm/phương thức (methods).
- ◆ Object: Một đối tượng cụ thể được tạo ra từ class.

a. Cách khởi tạo 1 Class cơ bản

```
class Student:
    def __init__(self):
        self.name = None
        self.age = None
        self.math_score = None
        self.literature_score = None
        self.english_score = None
    def avarage_score(self):
        ava_score = (self.math_score + self.literature_score + self.english_score)/3
        return ava_score

student1 = Student()
student1.name = "Thien Duong"
student1.age = 31
student1.math_score = 9
student1.literature_score = 8
student1.english_score = 9

print(student1.avarage_score())
```

Trong đó:

- name, age, math_score, literature_score, english_score được gọi là **thuộc tính** của class Student
- __init__(self) để khai báo tất cả các thuộc tính có trong class Student
- avarage_score(self) được gọi là hành động của class Student
- Biến student1 được truy cập đến class Student, và muốn gọi các thuộc tính thì có cú pháp student1.thuoc_tinh
- Biến student1 có thể overwrite như các biến bình thường



a. Cách khởi tạo 1 Class cơ bản

```
class Student:
    def __init__(self):
        self.name = None
        self.age = None
        self.math_score = None
        self.literature_score = None
        self.english_score = None
    def avarage_score(self):
        self.ava_score = (self.math_score + self.literature_score + self.english_score)/3
        return self.ava_score
    def rank(self):
        avarage = self.avarage_score()
        print("avarage score is : " + str(avarage))
        if avarage >= 9:
            print("A")
        elif 8 <= avarage < 9:
            print("B")
        elif 7 <= avarage < 8:
            print("C")
        else:
            print("D")
```

```
student1 = Student()
student1.name = "Thien Duong"
student1.age = 31
student1.math_score = 9
student1.literature_score = 8
student1.english_score = 9
student1.rank()
```

Trong đó:

- avarega_score(self) được gọi là hành động của class Student
- Có thể khởi tạo một hành động khác trong class Student, ví dụ là hành động rank(self) với mục đích xếp loại dựa trên điểm trung bình 3 môn học được tính từ hành động avarega_score(self)

a. Cách khởi tạo 1 Class cơ bản

Gọi thêm 1 biến khác là student2, và cũng truy cập các thuộc tính từ trong class Student.

```
student1 = Student()
student1.name = "Thien Duong"
student1.age = 31
student1.math_score = 9
student1.literature_score = 8
student1.english_score = 9
print(student1.average_score())

student2 = Student()
student2.name = "Ronaldo"
student2.age = 41
student2.math_score = 7
student2.literature_score = 9
student2.english_score = 10
print(student2.average_score())
```

Trong Python, **class (lớp)** là một **mô hình (blueprint)** để tạo ra các **đối tượng (objects)**. Lập trình hướng đối tượng giúp **quản lý dữ liệu và chức năng dễ dàng hơn** bằng cách gói gọn chúng vào một đối tượng.

- ◆ Class: Một khuôn mẫu chứa thuộc tính (attributes) và hàm/phương thức (methods).
- ◆ Object: Một đối tượng cụ thể được tạo ra từ class.

```
class Person:
    def __init__(self, name, age):
        self.name = name # Thuộc tính name
        self.age = age   # Thuộc tính age

    def introduce(self): # Phương thức / hành động
        print(f"Hello, my name is {self.name}, {self.age} yearold.")
```

Cách khai báo 1 class

```
p1 = Person("Duong Minh Thien", 31)
p1.introduce()
```

Cách khai báo 1 object

b. Cách khởi tạo 1 Class với các parameter/agurment

```
class Student:
    def __init__(self,name,age,math_score,literature_score,english_score):
        self.name = name
        self.age = age
        self.math_score = math_score
        self.literature_score = literature_score
        self.english_score = english_score
    def avarage_score(self):
        self.ava_score = (self.math_score + self.literature_score + self.english_score)/3
        return self.ava_score
    def rank(self):
        avarage = self.avarage_score()
        print("avarage score is : " + str(avarage))
        if avarage >= 9:
            print("Rank is : A")
        elif 8 <= avarage < 9:
            print("Rank is : B")
        elif 7 <= avarage < 8:
            print("Rank is : C")
        else:
            print("Rank is : D")
```

```
student1 = Student("Ronaldo", 21, 9, 7, 5)
student1.rank()
student2 = Student("Messi", 19, 5, 6, 8)
student2.rank()
student3 = Student("Mbape", 29, 10, 6, 7)
student3.rank()
```

c. Thực hiện hành động trong class bằng vòng lặp For

```
students = []

student1 = Student("Ronaldo", 21, 9, 7, 5)
student2 = Student("Messi", 19, 5, 6, 8)
student3 = Student("Mbape", 29, 10, 6, 7)

students.append(student1)
students.append(student2)
students.append(student3)

for i in range(len(students)):
    | students[i].rank()
```

Có thể ghi đè giá trị của thuộc tính trong class bằng cách gán nó bằng 1 giá trị mới (overwrite)

```
students = []

student1 = Student("Ronaldo", 21, 9, 7, 5)
student2 = Student("Messi", 19, 5, 6, 8)
student3 = Student("Mbape", 29, 10, 6, 7)

student2.math_score = 8 #overwrite

students.append(student1)
students.append(student2)
students.append(student3)

for i in range(len(students)):
    | students[i].rank()
```

```
class Student:
    def __init__(self, name, student_id, gpa):
        self.name = name
        self.student_id = student_id
        self.gpa = gpa

    def display_info(self):
        print(f"SV: {self.name}, MSSV: {self.student_id}, GPA: {self.gpa}")

student1 = Student(input("Name: "), input("MSSV: "), float(input("GPA: ")))

student1.display_info()
```

d. Tính kế thừa

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Animal speaks"

class Dog(Animal):
    def speak(self):
        return "GauGau!"

class Cat(Animal):
    def speak(self):
        return "MeoMeo!"

dog = Dog("Buddy")
print(dog.speak()) # Output: GauGau!

cat = Cat("Kitty")
print(cat.speak()) # Output: MeoMeo!
```

Kế thừa (Inheritance) cho phép một **class con** kế thừa thuộc tính và phương thức từ **class cha**.

d. Tính kế thừa

```
class FatherDog:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Gaugau"

class BabyDog(FatherDog):
    pass
    # def speak(self):
    #     return "GauGau!"

dog = BabyDog("Buddy")
print(dog.speak()) # Output: GauGau!
```


d. Tính kế thừa

```
class FatherDog:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Gaugau!"

    def food(self):
        return "bone"

class BabyDog(FatherDog):
    pass

dog = BabyDog("Buddy")
print(dog.speak()) # Output: GauGau!
print(dog.food()) # Output: bone
```

d. Tính kế thừa

```
class FatherDog:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Gaugau!"

    def food(self):
        return "bone"

class BabyDog(FatherDog):
    def age(self):
        return "2"

dog = BabyDog("Buddy")
print(dog.speak()) # Output: GauGau!
print(dog.food()) # Output: bone
print(dog.age()) # Output: 2
```

Exercise 1:

Sử dụng kiến thức về class, viết chương trình tính diện tích và chu vi **hình vuông**.

Tận dụng **tính chất kế thừa** của class, để tính diện tích và chu vi **hình thoi**.

Gợi ý:

```
class Square:
    def __init__(self, side):
        self.side = side
```

```
class Rhombus(Square):
    def __init__(self, side, diagonal1, diagonal2):
        super().__init__(side)
        self.diagonal1 = diagonal1
        self.diagonal2 = diagonal2
```

Đáp án:

```
class Square:
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

    def perimeter(self):
        return 4 * self.side

class Rhombus(Square):
    def __init__(self, side, diagonal1, diagonal2):
        super().__init__(side)
        self.diagonal1 = diagonal1
        self.diagonal2 = diagonal2

    def area(self):
        return (self.diagonal1 * self.diagonal2) / 2
```

```
side = float(input("Canh hình vuông: "))
square = Square(side)
print(f"Chu vi hình vuông: {square.perimeter()}")
print(f"Dien tích hình vuông: {square.area()} ")

diagonal1 = float(input("Duong cheo 1: "))
diagonal2 = float(input("Duong cheo 2: "))
rhombus = Rhombus(side, diagonal1, diagonal2)
print(f"Chu vi hình thoi: {rhombus.perimeter()}")
print(f"Dien tích hình thoi: {rhombus.area()} ")
```

Exercise 2:

Sử dụng kiến thức về **class**, **FileIO** và **list** để viết chương trình quản lý thư viện:

- Tạo **class Book** với các thuộc tính:

- a) title: tiêu đề sách
- b) author: tác giả
- c) ISBN: mã số sách

display_info(): in thông tin sách

- Tạo **class Library** để quản lý sách, gồm:

- a) add_book(book): thêm sách vào thư viện, lưu vào file data.txt
- b) remove_book(ISBN): xóa sách theo mã ISBN ra khỏi file data.txt
- c) search_book(title): tìm sách theo tiêu đề từ file data.txt và hiển thị thông tin sách

Homework:

Sử dụng kiến thức về class, FileIO và list để viết chương trình cho phép người dùng nhập thông tin các cầu thủ.

- (i) Người dùng nhập 1 số nguyên (tổng số cầu thủ). Mỗi cầu thủ sẽ có **chiều cao** và **cân nặng**
- (ii) Người dùng nhập thông tin từng cầu thủ từ bàn phím (**chiều cao** và **cân nặng**). Sau đó mọi thông tin sẽ được lưu vào **file data.txt**. Dòng đầu là **số lượng cầu thủ** đã được nhập thông tin, sau đó cứ mỗi 2 dòng tiếp theo sẽ là **chiều cao** và **cân nặng**.
- (iii) Đọc file data.txt và lưu video vào **1 list**.
- (iv) Đọc list được tạo ở bước trên và in thông tin các cầu thủ ra màn hình. **VD: cauthu1 có chiều cao 1m8,....**

Lưu ý:

- Sinh viên viết chương trình và chú thích ý nghĩa câu lệnh bên cạnh, vd **# tăng biến i lên 1 đơn vị.**
- Sinh viên viết chương trình và lưu riêng lẻ từng file .py ứng với mỗi câu hỏi, vd cau1.py. Sau đó, nén 3 file .py thành file zip/rar và đặt tên theo quy định: HoTen_MSSV_BTChuong6 và nộp lên trang utexlms
- Thời gian làm bài 1 tuần kể từ ngày giao bài tập.
- Các chương trình giống nhau với tỷ lệ bất thường sẽ không được chấm điểm
- SV nộp trễ với bất kỳ lý do gì cũng không được giải quyết
- **SV PHẢI NỘP KÈM CÁC FILE BT TRÊN LỚP CÙNG VỚI BT VỀ NHÀ LUÔN NHÉ!**



TRƯỜNG ĐẠI HỌC

SƯ PHẠM KỸ THUẬT TP.HỒ CHÍ MINH

HCMC University of Technology and Education

Thank you for
your listening

Q & A

